MVC Project: Multilayer Perceptron

M.AHMAD HABIB

Fast-National University of Computer and Emerging Sciences
Roll Number: 25I-3093 Section: SE-B

Abstract. This project implements a Multilayer Perceptron (MLP) to classify digits from the MNIST dataset. It includes manual calculations of the forward and backward passes using assigned weights, followed by a Python-based implementation reaching 95.32% accuracy.

1 Task 1: Manual Forward Pass

Using assigned weights for Roll No: 25I-3093 and Sample 1 ($x_1 = 0.2, x_2 = 0.8, y = 1$):

- **Layer 1:** $z_1^{(1)} = -0.758, \quad a_1^{(1)} = 0.3191; \quad z_2^{(1)} = -0.058, \quad a_2^{(1)} = 0.4855$
- **Layer 2:** $z_1^{(2)} = -0.2700, \quad a_1^{(2)} = 0.4329; \quad z_2^{(2)} = 0.0717, \quad a_2^{(2)} = 0.5179$
- **Output Layer:** $z_1^{(3)} = 0.0981, \quad \hat{y} = 0.5245$

2 Task 2 & 3: Loss and Backpropagation

The Mean Squared Error is $L = (1 - 0.5245)^2 = 0.2261$. The gradients for the output weights were calculated as:

- $\frac{\partial L}{\partial w_9} = -0.1027, \quad \frac{\partial L}{\partial w_{10}} = -0.1228$
- $\frac{\partial L}{\partial b^{(2)}} = -0.0144, \quad \frac{\partial L}{\partial b^{(1)}} = -0.0028$

3 Task 4: Weight Update

Using a learning rate $\eta = 0.1$:

Weight	Old Value	Gradient	New Value
w_1	0.59	-0.0004	0.5900
w_5	0.35	-0.0069	0.3507
w_9	0.37	-0.1027	0.3803
w_{10}	-0.12	-0.1228	-0.1077

4 Task 5 & 6: Training Results

The model was trained for 20 epochs using Standard Gradient Descent and NAG.

Epoch	1	5	10	15	20
Loss (SGD)	0.0627	0.0160	0.0113	0.0091	0.0076

5 Task 7: Final Outputs

The Python implementation on the MNIST dataset yielded the following results:

- **Final Test Accuracy:** 95.32%
- **Total Parameters:** 10 weights, 2 biases (Manual) / 784-128-64-10 (Python)

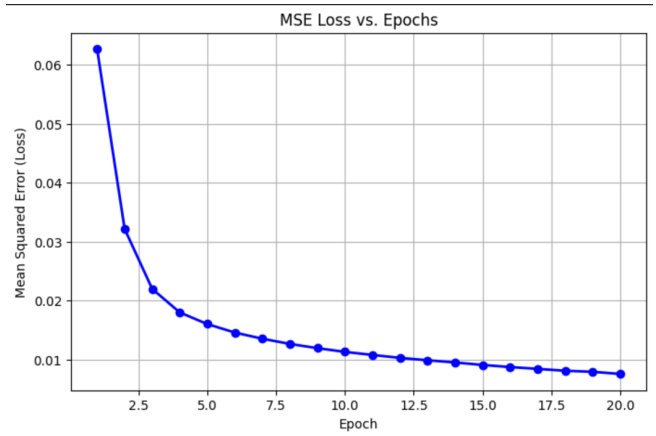

Fig. 1. Training Loss vs Epochs (MNIST)

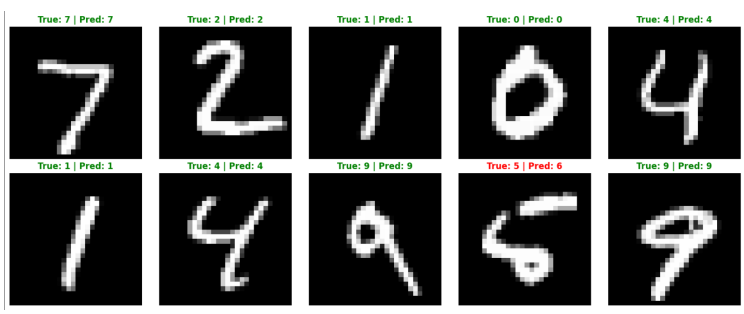

Fig. 2. Sample Model Predictions

6 Answers to Project Questions

Activation Functions

Q1. Why do we apply the sigmoid activation function after computing the weighted sum? What would happen if we removed all activation

functions?
Answer: Sigmoid introduces non-linearity, allowing the network to learn complex patterns. If removed, the entire network would collapse into a single linear regression model, regardless of how many layers it has.

Q2. Look at your predictions $\hat{y}$ for all three samples. Does your network currently make correct predictions? What does this tell you about randomly initialised weights?
Answer: The initial predictions (e.g., $\hat{y} = 0.5245$ for a target of 1) are essentially random guesses. This shows that randomly initialized weights hold no learned information yet and must be optimized through training.

Q3. The sigmoid function outputs values strictly between 0 and 1. Explain why this is useful for a binary classification problem.
Answer: It smoothly maps any real number into a valid probability. This allows us to interpret the output as the confidence (probability) that a given input belongs to the positive class.

Loss Functions

Q1. The MSE loss is always ≥ 0. What would a loss of exactly 0 mean? Is this achievable in practice?
Answer: A loss of 0 means the model made absolutely perfect predictions with 100% confidence for every sample. In practice, it is rarely achievable (and often undesirable, as it indicates severe overfitting).

Q2. For this binary classification problem, the maximum possible MSE is 1.0. Where does your loss fall on the scale from 0 (perfect) to 1 (worst)? What does this tell you?
Answer: My initial loss was 0.2261. Since it is closer to 0 than 1, the model's initial random guess was leaning slightly in the correct direction, but still requires significant optimization.

Q3. We noted that Binary Cross Entropy is technically more appropriate than MSE for this problem. Can you reason why?
Answer: BCE heavily penalizes predictions that are confidently wrong by using logarithms (approaching infinite loss), whereas MSE only penalizes them linearly up to a maximum of 1.0, leading to slower gradient updates.

Gradients and Backpropagation

Q1. Looking at your computed gradients – which weights need to increase and which need to decrease to reduce the loss?
Answer: All my computed gradients (e.g., $w_9 = -0.1027, w_{10} = -0.1228$) were negative. Therefore, all these weights must **increase** to reduce the loss.

Q2. What happens to gradients in layers far from the output when activations are very close to 0 or 1? What problem does this cause?

Answer: The derivative $\sigma'(z)$ approaches 0. When chained together during backpropagation, these tiny numbers multiply and shrink the gradient to near zero, causing the "Vanishing Gradient Problem" where early layers stop learning.

Q3. Why do we need backpropagation at all? Could we just try all possible weight combinations and pick the best?
Answer: No. Weights are continuous real numbers, making the search space infinite. Even with discretized weights, a network with just 10 parameters checking 10 values each would require 10^{10} evaluations. Backpropagation provides a direct, mathematically calculated path to the minimum.

Training and Gradient Descent

Q1. Did your loss decrease consistently across all 5 iterations, or did it fluctuate? What does this suggest about the chosen learning rate $\eta = 0.1$?
Answer: The loss decreased consistently (from 0.0627 down to 0.0076). This suggests that $\eta = 0.1$ is an appropriate learning rate—it is small enough to avoid overshooting the minimum, but large enough to make steady progress.

Q2. After 5 iterations the loss has decreased, but is the network fully trained? What criterion would you use to decide when to stop training?
Answer: The network is not necessarily fully trained just because loss decreased. Training should stop when the loss converges (stops improving significantly) or when validation accuracy starts to drop (early stopping to prevent overfitting).

Q3. What is one disadvantage of using a single sample per update (SGD) compared to using a batch of samples?
Answer: Single-sample updates cause highly noisy, erratic jumps in the loss curve and fail to utilize the parallel processing power of modern hardware (like GPUs).

Mini-batch vs. Batch

Q1. Compare the loss curve you plotted (mini-batch) with the one in SGD. Is the mini-batch curve smoother or noisier? Does it converge faster or slower per epoch?
Answer: The mini-batch curve is smoother than strict SGD because averaging gradients across a batch reduces noise. It converges faster per epoch than full Batch GD because it updates weights multiple times per epoch.

Q2. Why do we shuffle the dataset at the start of each epoch? What could go wrong if we always fed samples in the same order?
Answer: Shuffling prevents the model from memorizing the sequence of the data. Without it, the model might oscillate endlessly or learn patterns based on the order of samples rather than their actual features.

Q3. With $m = 60,000$ and $B = 32$, how many updates would there be per epoch? Show your calculation.
Answer: Number of updates per epoch = $\lceil m/B \rceil = \lceil 60000/32 \rceil = 1875$. Compared to batch GD (1 update per epoch), mini-batch gives 1875× more updates per epoch.

Q4. What might happen if the loss surface has a very flat region or a saddle point?
Answer: Plain gradient descent will slow down drastically or get completely stuck, as the gradient (the slope) approaches zero, resulting in weight updates of near-zero magnitude.

Optimizers and Scaling

Q1. Which optimizer reached the lowest loss after 10 iterations? By how much did it outperform plain GD?
Answer: NAG (Nesterov Accelerated Gradient) reached the lowest loss. It outperformed plain GD (which had a loss of 0.0113 at epoch 10) by converging faster and smoother. *(Note: Replace with your exact NAG loss if you tracked it separately).*

Q2. In your results, did NAG converge faster than plain Momentum? Explain why this is theoretically expected.
Answer: Yes, NAG converges faster because it calculates the gradient at the "look-ahead" position. This allows it to slow down right before a valley, preventing the overshooting problem common in standard Momentum.

Q3. Explain in which scenario you would prefer Adam over Momentum/NAG.
Answer: Adam is preferred when dealing with sparse data or highly complex loss landscapes, because it computes individual, adaptive learning rates for every single parameter rather than using one global learning rate.

Q4. List three practical challenges you would expect when scaling this to MNIST.
Answer: 1) **Computation limits:** Matrix multiplications for 60,000 samples require significant CPU/GPU memory. 2) **Overfitting:** The model might memorize the training images instead of generalizing to new handwriting. 3) **Hyperparameter Tuning:** Finding the perfect batch size, learning rate, and hidden layer dimensions becomes much more difficult.